Prototyping

Once the API specs are put in place, prototyping helps you visualize the API before actual development by letting the developers create MOCK API to help them understand every potential aspect of the API.

Authentication and Authorization

Authentication involves the verification process to know who the person is, but it just doesn't involve giving access to all the resources yet, and that's where authorization comes in, which involves authorizing an authenticated person to keep a check on resources allowed to access using an Access Control List (ACL).

We have different ways of authenticating and authorizing users like basic authentication, HMAC, and OAuth. OAuth 2.0 is however a preferred method for the same and is a standard protocol used by enterprises as well as small companies for authentication and authorization in their REST APIs.

So, these are the key features of the REST infrastructure, and we'll discuss more about how REST works and enables better communication in later chapters.

Now, we'll start with setting up our development environment and understand some key factors of developing applications with Python.

Setting Up Development Environment

In this part, we'll discuss setting up Python development environment for a Flask application. We'll use virtual environments for a separate isolated environment for our dependencies. We'll use PIP for installing and managing our dependencies and a couple of other helpful utilities in the process of setting up our development environment. For the sake of this book, we'll be doing everything on macOS Mojave and Python 2.7, but you

can feel free to use any operating system as per your convenience. So if you don't have the right version of Python installed in your operating system, you can go ahead with installing Python on your choice of operating system using this link: www.python.org/downloads/ (Figure 1-5).

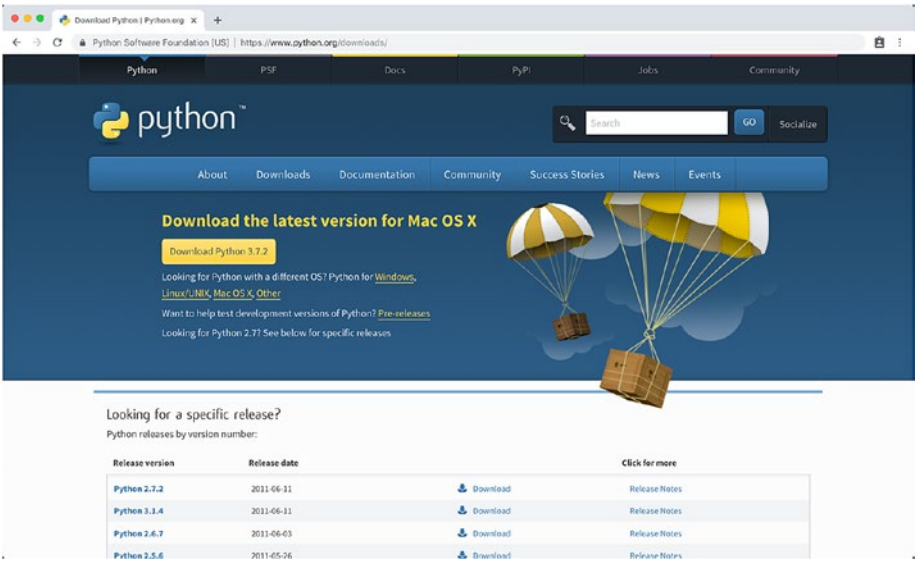

Figure 1-5. *Python download*

Working with PIP

PIP is a PyPi recommended tool for project dependency management. PIP comes preinstalled with Python if you are using Python downloaded from www.python.org.

However, if you don't have PIP installed in your system, follow the guide here to install PIP.

In order to install PIP, download get-pip.py by using the following command in your terminal (or command line in Windows).

```
$ curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

Once you have the get-pip.py file, install and run the next command:

```
$ python get-pip.py
```

The previous command will install PIP, setuptools (required for installing source distributions), and wheel.

If you already have PIP, you can upgrade to the latest version of pip using the following command:

```
$ pip install -U pip
```

To test your installation, you should run the following command (Figure 1-6) in your terminal (or command line in Windows):

```
$ python -V
$ pip -V
```

Figure 1-6. *Checking Python and PIP installation*

Choosing the IDE

Before we start writing the code, we'll need something to write with. Throughout this book, we'll use Visual Studio Code which is an open source and free IDE available on all major operating systems. Visual Studio Code is available to download from `www.code.visualstudio.com`, and it provides good support for developing Python applications with plenty of handy plugins to facilitate development. You can choose to use your own preferred text editor or IDE to follow this book (Figure 1-7).

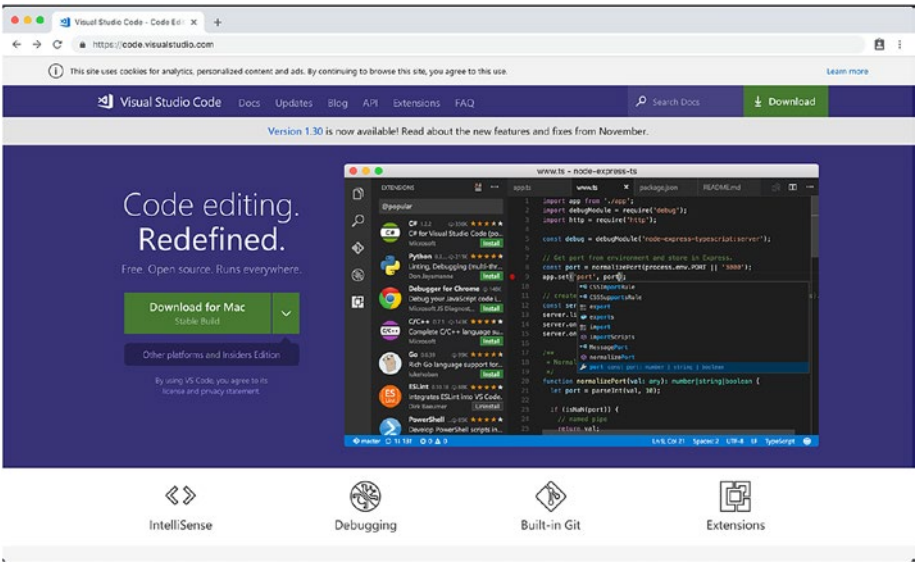

Figure 1-7. *Visual Studio Code*

Once we have the IDE setup, we can move to installing and setting up the virtual environment.

Understanding Python Virtual Environments

Python, just like other modern programming languages, provides a huge amount of third-party libraries and SDKs. Different applications might need various specific versions of third-party modules, and it won't be possible for one Python installation to meet such requirements of every application. So, in the world of Python, the solution for this problem is virtual environment, which creates a separate self-contained directory tree containing a Python installation of the required version alongside the required packages.

At its core, the main purpose of a virtual environment is to create an isolated environment to contain an installation of Python and required packages for the application. There is no limit to the number of virtual environments you can create, and it's super easy to create them.

Using Virtual Environments

In Python 2.7 we need a module called **virtualenv** which is installed using PIP to get started with Python virtual environments.

Note In Python 3 the venv module comes preshipped as a part of the standard library.

To install virtualenv, type the following command in your terminal (or command line in case of Windows).

```
$ pip install virtualenv
```

Once we have the virtualenv module installed in our system, next we'll create a new directory and create a virtual environment in it.

Now, type the following command to create a new directory and open it in your terminal.

```
$ mkdir pyenv && cd pyenv
```

The preceding command will create a directory and open it in your terminal, and then we'll use the virtualenv module to create a new virtual environment inside the directory.

```
$ virtualenv venv
```

The previous command will use the virtualenv module and create a virtual environment called venv. You can name your virtual environment anything, but for this book, we'll just use venv for the sake of uniformity.

Once this command stops executing, you'll see a directory called venv. This directory will now hold your virtual environment.

The directory structure of the venv folder should be similar to the one in Figure 1-8.

```
Kunals-MacBook-Pro:venv kunalrelan$ tree -L 2
.
├── bin
│   ├── activate
│   ├── activate.csh
│   ├── activate.fish
│   ├── activate_this.py
│   ├── easy_install
│   ├── easy_install-2.7
│   ├── pip
│   ├── pip2
│   ├── pip2.7
│   ├── python
│   ├── python-config
│   ├── python2 -> python
│   ├── python2.7 -> python
│   └── wheel
├── include
│   └── python2.7 -> /System/Library/Frameworks/Python.framework/Versions/2.7/include/python2.7
├── lib
│   └── python2.7
└── pip-selfcheck.json

5 directories, 15 files
Kunals-MacBook-Pro:venv kunalrelan$
```

Figure 1-8. *Virtual environment directory structure*

Here is what each folder in the structure contains:

1. bin: Files to interact with the virtual environment.
2. include: C headers to compile the Python packages.
3. lib: This folder contains a copy of the Python version and all the other third-party modules.

Next, there are copies of, or symlinks to, different Python tools to ensure all the Python code and commands are executed within the current environment. The important part here is the activation scripts in bin folder, which sets the shell to use the virtual environment's Python and site packages. In order to do so, you need to activate the virtual environment by typing the following command in your terminal.

```
$ source venv/bin/activate
```

Once this command is executed, your shell prompt will be prefixed with the name of the virtual environment, just as in Figure 1-9.

```
Kunals-MacBook-Pro:pyenv kunalrelan$ source venv/bin/activate
(venv) Kunals-MacBook-Pro:pyenv kunalrelan$
```

Figure 1-9. *Activating virtual environment*

Now, let's install Flask in our virtual environment using the following command:

```
$ pip install flask
```

The preceding command should install Flask in our virtual environment. We'll use the same code we did in our sample Flask application.

```
$ nano app.py
```

And type the following code in the nano text editor:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, From Flask!'

Now,  try running your app.py using python app.py command.

$ FLASK_APP=app.py flask run
```

With the preceding command, you should be able to run the simple Flask application, and you should see similar output in your terminal (Figure 1-10).

```
(venv) Kunals-MacBook-Pro:pyenv kunalrelan$ FLASK_APP=app.py flask run
 * Serving Flask app "app.py"
 * Environment: production
   WARNING: Do not use the development server in a production environment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Figure 1-10. *Running Flask application in virtual environment*

Now, to deactivate the virtual environment, you need to execute the following command:

```
$ deactivate
```

After this command executes, (venv) prefix from the shell will go away, and if you try running the application again, it will throw an error (Figure 1-11).

```
Kunals-MacBook-Pro:pyenv kunalrelan$ FLASK_APP=app.py flask run
-bash: flask: command not found
Kunals-MacBook-Pro:pyenv kunalrelan$
```

Figure 1-11. *Running Flask application without virtual environment*

So now you understand the concept of virtual environments, we can dig a little deeper and understand what's happening inside the virtual environment.

Understanding how virtual environments work can really help you debug the application and understand the execution environment. To start with, let's check out the Python executable with virtual environment activated and deactivated, in order to understand the basic difference.

Let's execute the following command with virtual environment activated (Figure 1-12):

```
$ which python
```